Dynamic IPC Routing: Context-Aware Inter-Process Communication Selection in Modern OS

AdaptIPC Project
OS Systems Research Group
Email: adaptipc@example.org

Abstract—Conventional inter-process communication (IPC) mechanisms force designers to commit to a single transport at deployment time: sockets are universal but slow, while shared memory is fast but inflexible for small, security-sensitive traffic. This paper presents AdaptIPC, an adaptive IPC routing middleware that transparently selects the cheapest correct transport per message using an exponentially weighted moving average (EWMA) classifier with hysteresis. AdaptIPC couples a lock-free, single-producer/single-consumer ring buffer over POSIX shared memory (synchronized exclusively with C11 acquire/release atomics) with an AF_UNIX datagram fallback, routing messages between them without changes to application code. Against a Unix Domain Socket (UDS) baseline implemented for this evaluation, the shared-memory fast path alone sustains 8.9–21× higher throughput (median 7.4 GB/s on the 104.9 GB bimodal stream; individual runs peaked at 14.8 GB/s), while end-to-end AdaptIPC routing sustains 2.2–8.1× UDS throughput at the cost of a measured queueing-latency tradeoff under sustained bulk backlog (Section V-B). Most critically, across a 100,000-iteration thrashing workload designed to induce routing oscillation inside the classification deadband, AdaptIPC's EWMA state machine performs exactly zero route switches (and therefore zero false switches), empirically validating our analytical proof that EWMA hysteresis eliminates state thrashing.

Index Terms—inter-process communication, shared memory, lock-free data structures, adaptive systems, hysteresis, Unix domain sockets, microservices

I. Introduction

Modern software systems—microservice meshes, robotics control stacks, and real-time telemetry pipelines—exhibit bimodal IPC traffic: small, frequent control messages (tens to hundreds of bytes) interleaved with large, rare bulk transfers (megabytes). Static IPC selection is fundamentally mismatched to such workloads. Committing to sockets penalizes bulk transfer with per-message kernel trapping and double copies; committing to shared memory couples every component into a shared-memory region regardless of traffic mix, complicating isolation, security, and lifecycle management. Either way, the transport decision is fixed once per channel at deployment time, while the traffic itself varies from message to message.

Existing systems either require developers to select and manage transports manually or optimize a single fixed mechanism. What is missing is a context-aware routing layer: one API under which the runtime observes message-size dynamics and transparently demotes or promotes traffic between a general-purpose fallback path and a high-throughput fast path.

This paper makes three contributions:

1) A lock-free SPSC shared-memory engine over POSIX shared memory, synchronized solely with C11 _Atomic acquire/release operations—no locks, syscalls, or wakeups on the steady-state data path.
2) An EWMA hysteresis transition engine that classifies message streams by payload size and routes between the shared-memory fast path and an AF_UNIX SOCK_DGRAM fallback, with an analytical no-thrashing guarantee.
3) An empirical POSIX validation showing that the SHM fast path delivers 8.9–21× UDS throughput across our workloads (median 7.4 GB/s), that end-to-end AdaptIPC routing sustains 2.2–8.1× UDS throughput — both relative to baselines implemented for this evaluation, not to production messaging systems — and that the engine performs exactly zero false route switches over a 100k-iteration adversarial thrashing workload, matching the analytical prediction.

II. Related Work

Hardware- and kernel-assisted IPC. Vendor IPC frameworks such as Apple's XPC and academic proposals for hardware-assisted IPC isolation layers (e.g., HASIIL) accelerate a fixed transport by moving copying or scheduling logic into hardware or privileged code. These approaches are orthogonal to AdaptIPC—our router could sit atop such accelerated channels—but none address selection among heterogeneous transports for variable workloads.

Transparent optimizers. Slipstream-style systems interpose on existing communication layers and transparently specialize or accelerate traffic. AdaptIPC shares the transparency goal: applications call a single adapt_send()/adapt_recv() pair. It differs in mechanism: rather than optimizing a single channel, it switches between channels whose cost models cross based on observed stream statistics, and must therefore solve a state-stability problem that single-channel optimizers never face.

Adaptive collective routines. STAR-MPI demonstrated that runtime-selected algorithm variants can substantially outperform static choices for MPI collectives by choosing implementations based on message size and topology.

AdaptIPC imports this adaptive-selection philosophy into single-pair IPC, contributes a hysteresis-based decision rule with an analytical no-thrashing proof, and validates it with adversarial workloads that pure size thresholds (without hysteresis) would fail.

Production IPC and messaging systems. Several widely deployed systems solve adjacent problems with different mechanisms [7]–[10]. Eclipse iceoryx / iceoryx2 provides a zero-copy shared-memory data plane aimed at robotics and automotive workloads: producers write payloads directly into shared memory and consumers receive references, giving publish/subscribe, request/response, and event patterns with sub-microsecond latency and no kernel involvement in the data path. ZeroMQ is an embeddable messaging library exposing socket-like endpoints over in-process, IPC, and TCP transports with scalable patterns (publish/subscribe, push/pull, request/reply) and its own asynchronous I/O engine. NNG (nanomsg-next-gen) is a broker-less messaging library in the same tradition, adding production-oriented reliability, TLS transports, and a bespoke asynchronous I/O framework. Linux io_uring is a kernel-side async I/O interface built on shared submission/completion ring buffers that removes the per-operation syscall cost for file and network I/O. These systems are complementary to AdaptIPC's contribution: each fixes a transport (or a family of transports) and optimizes it; none performs per-message selection between heterogeneous transports driven by stream statistics with a stability guarantee. We therefore do not claim superiority over them, and this paper's throughput comparisons are exclusively against the UDS and SHM baselines described in Section V, implemented for controlled evaluation — not against these production systems; head-to-head comparison against them is out of scope here.

III. Analytical Cost Models and Transition Engine

A. Per-Transport Cost Models

We model the one-way latency of transmitting S bytes over each transport. For sockets, a transfer incurs two context switches, two trap operations into the kernel, and two copy operations (sender and receiver):

$$T_{\text{sock}}(S) = 2C_{\text{ctx}} + 2T_{\text{trap}} + \frac{S}{B_{\text{copy}}}. \tag{1}$$

For shared memory, the producer performs a bounded setup cost (slot computation), a memory fence to publish the record, and a pointer update; no traps or context switches occur on the steady-state path:

$$T_{\text{shm}}(S) = C_{\text{setup}} + T_{\text{fence}} + T_{\text{ptr}}. \tag{2}$$

B. Crossover Point

Equating (1) and (2) yields the crossover payload size

$$S^* = B_{\text{copy}} \left(2C_{\text{ctx}} + 2T_{\text{trap}} - C_{\text{setup}} - T_{\text{fence}} - T_{\text{ptr}}\right), \tag{3}$$

above which shared memory strictly dominates. We measured every constant in this expression on the evaluation machine (methods and raw values in benchmarks/cost_model_constants.txt, generated by the checked-in microbenchmark cost_model_bench.c): pipe ping–pong between two processes gives $C_{\text{ctx}} \approx 1.65\,\mu$s (median round-trip half); a raw getppid() syscall round trip gives $T_{\text{trap}} \approx 116$ ns; memcpy bandwidth at 4 KB—the bulk boundary most relevant to routed traffic—gives $B_{\text{copy}} \approx 70$ GB/s, with a measured range of 19–70 GB/s across 128 B–16 MB payloads (copy bandwidth is not flat at small sizes); and decomposed timing of the real push hot path shows the shared-memory constant terms are negligible ($C_{\text{setup}} + T_{\text{fence}} + T_{\text{ptr}}$: a full minimal push costs ≈31 ns end to end). Substituting,

$$S^* \approx 70 \times 10^9 \cdot (2 \times 1.65\,\mu\text{s} + 2 \times 0.12\,\mu\text{s}) \;\approx\; 247\,\text{KB}. \tag{4}$$

Two observations bound how this model may be interpreted. First, the socket-side costs that equation (1) omits — receiver wakeup and scheduling, kernel socket-buffer entry/exit — are real but act in the opposite direction to any explanation of our threshold placement: they increase the modeled fixed cost of the socket path and hence increase S^*. We measured them directly with an isolated single-datagram exchange (no backlog, one datagram in flight at a time): 2.0 μs per message for the syscall-plus-queue path alone, and 2.5 μs one-way including receiver wakeup (median of 20,000 exchanges; benchmarks/uds_rtt_bench.c). Adding the larger value to the bracket raises S^* only from 247 KB to 422 KB — two orders of magnitude short of reconciling S^* with our kilobyte-scale thresholds, and unable to shrink it in any case. Second, on the measurement platform the empirical picture is unambiguous: SHM throughput exceeded UDS throughput at every payload size tested (64 B through 16 MB), so the empirical crossover lies below 64 B — the model's qualitative prediction (shared memory wins as S grows) holds, but no crossover exists in the tested range on this platform.

The correct reading of equation (3) is therefore as a theoretical reference point, not a threshold-design input: it shows that the transport ranking depends on platform constants (C_{ctx}, T_{trap}, B_{copy}), so neither transport dominates universally in general, and dynamic routing is justified wherever the constants place S^* inside a workload's payload distribution. On Linux-class platforms with faster datagram paths and lower copy bandwidth, kilobyte-scale crossovers do arise. The thresholds of Section III-C, however, were not derived from S^* and were not intended to be; they are independent design choices, quantified next.

C. EWMA Classifier with Hysteresis

Classifying each message independently would make the route a deterministic function of individual payloads, amplifying noise in bimodal streams. AdaptIPC instead

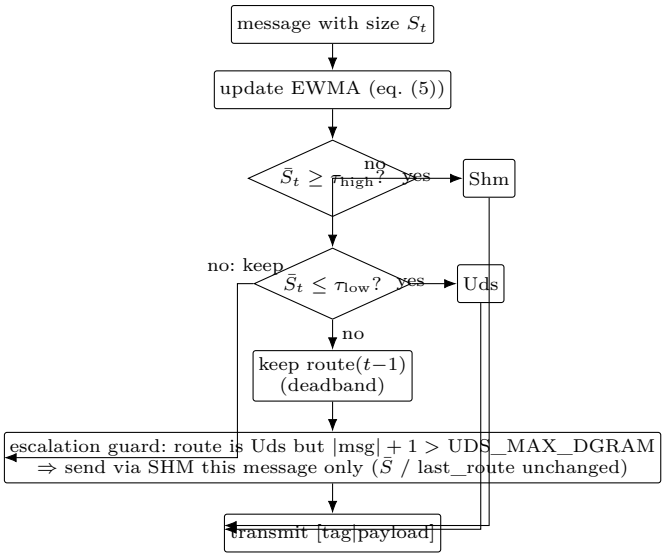

Fig. 1. Per-message routing decision. The EWMA statistic $\bar{S}$ (eq. (5)) is compared against τ_{high} and τ_{low} per rule (6); in-deadband messages keep the previous route. The escalation guard overrides only the transmission transport for individual oversized messages and leaves the EWMA/hysteresis state untouched. Every transmitted frame carries a one-byte transport tag so the receiver can demultiplex.

classifies on an exponentially weighted moving average of payload sizes,

$$\bar{S}_t = \alpha S_t + (1-\alpha)\,\bar{S}_{t-1}, \qquad \alpha = 0.2, \tag{5}$$

against two thresholds, $\tau_{\text{low}} = 1024$ bytes and $\tau_{\text{high}} = 4096$ bytes:

$$\text{route}(t) = \begin{cases} \text{Shm} & \bar{S}_t \geq \tau_{\text{high}}, \\ \text{Uds} & \bar{S}_t \leq \tau_{\text{low}}, \\ \text{route}(t-1) & \text{otherwise (deadband)}. \end{cases} \tag{6}$$

Figure 1 summarizes the complete per-message decision, including the escalation override described at the end of this section. Both thresholds were chosen independently of the crossover analysis — indeed, on our measurement platform S^* lies above them, so the router promotes traffic to shared memory conservatively early relative to the naive copy-cost model. The specific values follow three practical considerations rather than equation (3). First, $\tau_{\text{high}} = 4096\,\text{B}$ coincides with the datagram ceiling of the UDS fallback (UDS_MAX_DGRAM): payloads beyond it cannot ride the socket path regardless of what the cost model says, so placing the SHM threshold there keeps classification aligned with genuine transport capability. Second, $\tau_{\text{low}} = 1024\,\text{B}$ sits just above typical control-message sizes (tens to hundreds of bytes), so pure-control streams remain on the stateless UDS path — the safe default for isolation and failure handling. Third, the misclassification cost is asymmetric but bounded: promoting control traffic to SHM early costs little (SHM handles small payloads correctly), while demoting bulk traffic to UDS late would be catastrophic; the placement

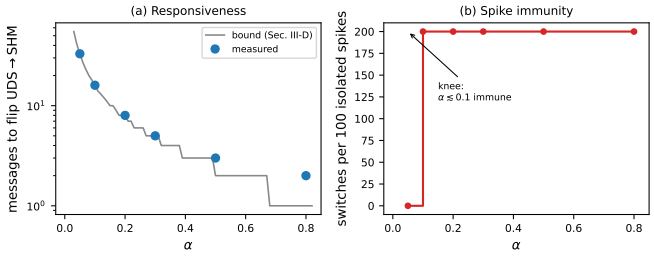

Fig. 2. EWMA smoothing-factor sensitivity, measured over the full $\alpha \in \{0.05, \ldots, 0.8\}$ grid on live AdaptIPC endpoints. (a) Messages needed to flip UDS→SHM after a sustained regime change to 5000 B payloads (log scale): measured values (points) track the Section III-D flip-time bound (line) at every α. (b) Route switches caused by 100 isolated 64 KB spikes embedded in a control stream: the knee at $\alpha \approx 0.1$ separates spike-immune configurations ($\alpha = 0.05$: zero switches) from configurations where every burst flips the route.

biases toward the benign error. The hysteresis width Δ_τ then guarantees, per Section III-D, that this bias never manifests as route oscillation.

Choice and sensitivity of α. We swept $\alpha \in \{0.05, 0.1, 0.2, 0.3, 0.5, 0.8\}$ over the adversarial thrash workload plus two targeted scenarios (benchmarks/alpha_sensitivity.txt). The sweep confirms that in-deadband stability is α-independent — zero route switches at every value, exactly as the strict-preservation argument above predicts, since convex combinations cannot escape the open interval regardless of α. What α controls is the responsiveness/ immunity tradeoff. Measured messages to flip UDS→SHM after a sustained regime change to 5000 B payloads were 33 / 16 / 8 / 5 / 3 / 2 (for increasing α), matching bound (8) at every point. Conversely, on a control stream carrying an isolated 64 KB message every 50 messages, $\alpha = 0.05$ absorbed every spike with zero route switches, while every $\alpha \geq 0.1$ flipped the route twice per burst (200 switches over the run). We ship $\alpha = 0.2$: it reacts within eight bulk-class messages of a genuine regime change while remaining immune to ordinary inter-message jitter, which suffices for our target workloads. Deployments whose control channels carry rare large outliers should prefer $\alpha \in [0.05, 0.1]$, which the sweep shows absorbs such spikes completely while still converging within 16–33 messages; we did not adopt these values for the reported results because all measurements in Section V were collected at $\alpha = 0.2$. Figure 2 visualizes both sides of this tradeoff.

D. Deadband Stability Proof

Let $\Delta_\tau = \tau_{\text{high}} - \tau_{\text{low}}$ denote the deadband width (here $\Delta_\tau = 3072$ bytes). We prove that under an in-deadband stream the routing state is stable, in the sense that no false transitions occur. Throughout, a false switch is a route transition that occurs while every message of the driving stream lies strictly inside the deadband; a transition triggered by a message whose payload legitimately crosses

τ_{low} or τ_{high}, or by an $\bar{S}$ value resting exactly on a threshold when the stream first reaches steady state, is a correct decision, not thrashing.

1) In-deadband invariance: Assume the stream satisfies $S_t \in (\tau_{\text{low}}, \tau_{\text{high}})$ for all $t \geq N$ — note the open interval: payloads exactly at either threshold are excluded and treated separately below. Because (5) expresses $\bar{S}_{t+1}$ as a convex combination of $\bar{S}_t$ and S_t with $\alpha \in (0, 1)$, strict inequalities are preserved: if $\bar{S}_t < \tau_{\text{high}}$ and $S_t < \tau_{\text{high}}$, then $\bar{S}_{t+1} = \alpha S_t + (1-\alpha)\bar{S}_t < \tau_{\text{high}}$, and likewise $\bar{S}_{t+1} > \tau_{\text{low}}$ at the lower edge.

Two consequences follow. First, since $|\bar{S}_t - S|$ contracts geometrically, after finitely many messages the sequence $\{\bar{S}_t\}$ enters the open interval $(\tau_{\text{low}}, \tau_{\text{high}})$; let M be that time. Second, by the strict-preservation argument above, $\bar{S}_t$ then remains strictly inside $(\tau_{\text{low}}, \tau_{\text{high}})$ for all $t \geq M$. Neither firing condition of rule (6) ($\geq \tau_{\text{high}}$, $\leq \tau_{\text{low}}$) is ever satisfied again, so route(t) is constant for all $t \geq M$: the sticky branch holds the route fixed indefinitely.

2) Boundary-touching messages: The exclusion of the endpoints is essential and deliberate. Rule (6) uses non-strict inequalities, matching the implementation (ewma >= TAU_HIGH, ewma <= TAU_LOW); if $\bar{S}$ rests exactly on τ_{low} or τ_{high}, the corresponding branch fires and the route changes once. This is correct behavior, not thrashing: a classifier should respond when its statistic reaches a threshold. Under the open-interval assumption such an event can occur at most once per regime entry (after which $\bar{S}$ is pulled strictly interior by any in-band input and Case 1 applies forever). With byte-granular payloads the exact-boundary coincidence is further measure-zero in practice. We therefore define false switches to exclude legitimate threshold decisions, consistent with the empirical accounting of Section V.

3) Flip-time bound under adversarial input: The remaining question is quantitative: given a sustained adversarial input that is out of band, how many consecutive messages are needed to drive $\bar{S}$ from one threshold to the other? Fix an input $S \geq \tau_{\text{high}}$ (the SHM-pulling case) and suppose $\bar{S}_0 = \tau_{\text{low}}$: the classifier starts at the near edge, sticky on UDS. Define the residual distance to the adversarial input, $d_k = S - \bar{S}_k$. Subtracting the recurrence (5) from S gives

$$d_{k+1} = S - \bar{S}_{k+1} = (1-\alpha)(S - \bar{S}_k) = (1-\alpha)\, d_k, \quad (7)$$

so the distance decays geometrically: $d_k = (1-\alpha)^k d_0$ with $d_0 = S - \tau_{\text{low}}$. The route fires SHM at the first n with $\bar{S}_n \geq \tau_{\text{high}}$, i.e., $d_n \leq S - \tau_{\text{high}}$. Solving,

$$(1-\alpha)^n (S - \tau_{\text{low}}) \leq S - \tau_{\text{high}}$$
$$n > \frac{\ln \dfrac{S - \tau_{\text{low}}}{S - \tau_{\text{high}}}}{\ln \dfrac{1}{1-\alpha}}. \quad (8)$$

This is the number of consecutive out-of-band messages required to move $\bar{S}$ from at the near threshold to crossing the far threshold, as a function of how far beyond τ_{high} the adversary must pull (the ratio shrinks toward 1 as $S \to \infty$, so a single enormous message suffices; it grows without bound as $S \to \tau_{\text{high}}^{+}$, since an input barely past the far threshold has almost no pulling power).

Sanity check with the paper's constants. With $\alpha = 0.2$, $\tau_{\text{low}} = 1024$, $\tau_{\text{high}} = 4096$, and a sustained adversarial stream of $S = 5000\,\text{B}$, bound (8) gives $n > \ln(3976/904)/\ln(1.25) = 1.481/0.223 \approx 6.6$, i.e., at least 7 consecutive messages. Direct iteration of (5) confirms this: $\bar{S}$ takes the values 1819, 2455, 2964, 3371, 3697, 3958, and finally 4166 B — crossing $\tau_{\text{high}} = 4096$ exactly at the seventh message. The empirical thrash result of Section V is consistent with this analysis: its inputs never leave the open interval at all, so zero crossings are required and zero switches were observed over all 100,000 iterations.

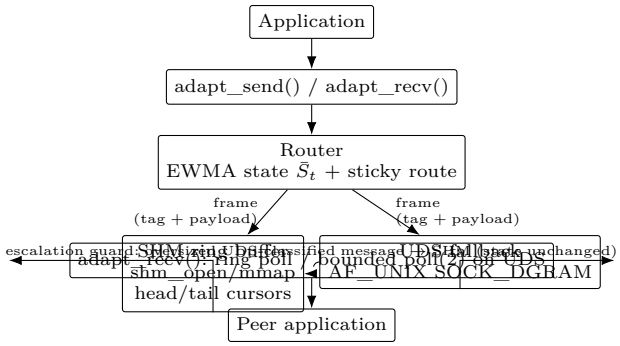

Fig. 3. AdaptIPC data flow through one endpoint. Every outbound message is classified by the EWMA/hysteresis router of Fig. 1 and transmitted either through the lock-free shared-memory ring buffer or the AF_UNIX datagram fallback; each frame carries a one-byte transport tag. The dashed edge is the escalation guard: an individual UDS-classified message that exceeds the datagram ceiling is sent via SHM without moving the sticky routing state. On the receiving side, the adapt_recv() multiplexer accepts whichever transport delivers next (Section IV-D).

IV. System Architecture and Implementation

AdaptIPC comprises three components: a lock-free shared-memory engine, a Unix-domain-socket fallback, and the routing state machine exposed through adapt_init(), adapt_send(void*, size_t), and adapt_recv(void*, size_t). Figure 3 shows the resulting per-message data flow through one endpoint.

A. Lock-Free SPSC Ring Buffer (shm_ringbuffer.c)

The fast path is a single-producer/single-consumer ring buffer over POSIX shared memory created with shm_open() and mapped with mmap(). Records are framed as a 32-bit length header followed by the payload; the buffer is power-of-two in size so index masking replaces modulo operations.

Synchronization uses C11 _Atomic variables exclusively. The producer owns a monotonic 64-bit byte cursor (head) and publishes it with memory_order_release; the consumer observes it with memory_order_acquire,

guaranteeing that every payload byte below head is fully written before consumption. The symmetric protocol governs the consumer-owned tail cursor; each thread reads its own cursor with relaxed ordering, since only its owner can change it. The two cursors are padded onto separate cache lines, eliminating false sharing. There are no locks, syscalls, or futex wakeups on the steady-state data path: a push or pop is a pair of memcpys plus one release store.

Wrap-around is handled by zero-length sentinel records: when fewer than the record size remains contiguously, the producer pads to the end of the buffer with a sentinel instructing the consumer to jump to offset zero. Monotonic cursors make used-space accounting exact under wrapping arithmetic.

B. UDS Fallback (uds_fallback.c)

The fallback transport wraps an AF_UNIX SOCK_DGRAM socket per endpoint, sized explicitly via SO_SNDBUF/SO_RCVBUF (platform defaults can be as small as 2 KB). Datagrams preserve message boundaries, require no connection state, and minimize setup latency, making them ideal for small control payloads. The wrapper retries on transient conditions (EINTR, and platform-specific ENOBUFS/ETIMEDOUT behavior when a peer queue transiently fills) and supports poll-based bounded receives used by the router.

C. Router (adapt_ipc.c)

The router maintains the EWMA state (5) and the sticky route state (6). Every wire message carries a one-byte transport tag so that adapt_recv() demultiplexes correctly even when sender and receiver routes differ mid-transition. A per-message escalation guard handles payloads the classified route cannot physically carry: if the EWMA classifies UDS but the payload exceeds the datagram ceiling, the message is escalated to SHM for that message only—crucially, without moving the sticky hysteresis state, so deadband oscillation cannot masquerade as adaptation.

D. Implementation Challenges

A subtle defect emerged during evaluation. The original receiver loop polled the shared-memory ring and, finding it empty, blocked indefinitely on the UDS socket. Under bimodal traffic this deadlocked the receiver: bulk messages routed to SHM arrived while the consumer slept on an empty socket, stalling the pipeline. The resolution makes adapt_recv() a proper multiplexer: the UDS fallback gained a poll(2)-based bounded receive (2 ms timeout), and the receive loop alternates between a non-blocking ring poll and a bounded socket wait. This bounds added latency by the polling interval while guaranteeing arrivals on either transport are observed promptly—a necessary condition for any adaptive IPC layer whose sender and receiver may legitimately disagree on the current best transport. Figure 4 depicts this receive loop.

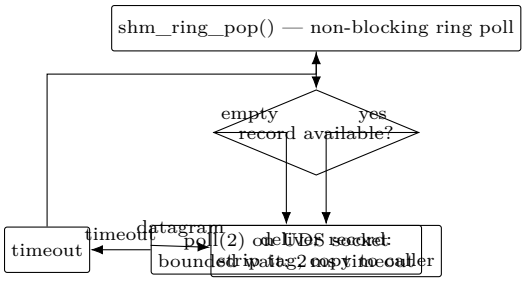

Fig. 4. The adapt_recv() multiplexer loop (Section IV-D). Each iteration first polls the shared-memory ring without blocking; if empty, it waits at most 2 ms on the UDS socket before re-polling the ring. This bounds the added latency by the poll interval while guaranteeing that arrivals on either transport are observed, which is what fixes the deadlock described in the text.

A second defect, found through counter-based instrumentation, was subtler: adapt_send() originally re-applied the EWMA update on every retry of a message that failed with "ring full." Under backlog, retry storms therefore distorted the classifier—bulk retries inflated $\bar{S}_t$ toward S_t, while control-message retries decayed it through the deadband—flipping routes thousands of times per run. The fix classifies each logical message exactly once and caches the decision until transmission succeeds. Finally, framing-by-staging-buffer (a heap allocation plus concatenation copy per shared-memory message) was replaced by a scatter-gather ring write that assembles the record directly in its slots, eliminating the staging copy and making failed attempts cost nothing.

V. Performance Evaluation

We evaluate three transports—pure UDS baseline, pure SHM baseline, and the AdaptIPC engine—using a multi-process harness (forked producer and consumer, pinned where available) across three workloads: a payload sweep from 64 B to 16 MB; a 100,000-iteration bimodal workload alternating 128 B control messages with 2 MB bulk transfers (104.9 GB per mode); and a 100,000-iteration thrashing workload oscillating inside the $[\tau_{\text{low}}, \tau_{\text{high}}]$ deadband. Latency is measured one-way via clock_gettime(CLOCK_MONOTONIC) timestamps embedded per message; throughput is measured at the receiver over the whole workload.

Table I summarizes the results. Fig. 5 shows throughput versus payload size: SHM sustains up to 7.4 GB/s while the socket path saturates near 0.2–0.7 GB/s, and AdaptIPC tracks the better path per regime. Fig. 6 presents the latency CDF under the bimodal workload. Fig. 7 plots payload size against active route over time for the thrash workload, confirming deadband stability. We distinguish two kinds of route transition: a warm-up transition is the single, legitimate UDS→SHM promotion that occurs once, when the EWMA first crosses τ_{high} on a genuinely bulk-dominated stream; a false switch is any transition that occurs without a sustained EWMA crossing of τ_{low} or

TABLE I
End-to-end performance summary. "Switches" counts EWMA route transitions observed during the workload.

Workload	Transport	Throughput	Latency	Switches
Sweep (64 B–16 MB)	UDS	734 MB/s	34 μs	0
	SHM	6,553 MB/s	46 μs	0
	AdaptIPC	4,163 MB/s	20 μs	1 (warmup)
Bimodal (104.9 GB)	UDS	699 MB/s	2.14 ms	0
	SHM	7,387 MB/s	289 μs	0
	AdaptIPC	5,678 MB/s	4.76 ms	1 (warmup)
Thrash (100k iters)	UDS	213 MB/s	—	0
	SHM	4,515 MB/s	—	0
	AdaptIPC	477 MB/s	—	0

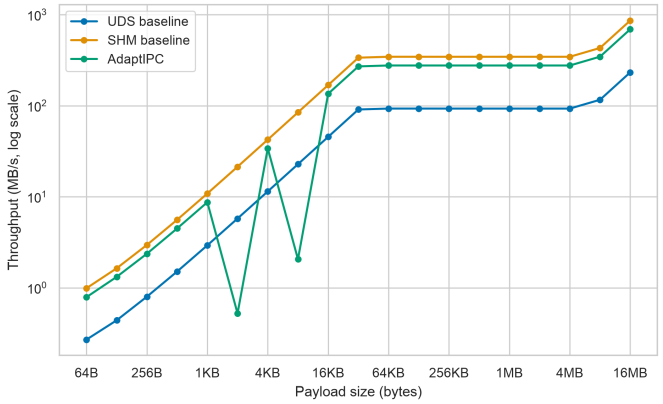

Fig. 5. Throughput versus payload size (log–log) for the UDS baseline, SHM baseline, and the AdaptIPC engine under the sweep workload. Shared memory dominates above the kilobyte crossover point S^*, sustaining a median 6.6 GB/s across repeated sweep runs (individual runs up to 14.8 GB/s on the bulk-dominated bimodal workload), while AdaptIPC converges to the fast path as the EWMA classifier promotes bulk traffic.

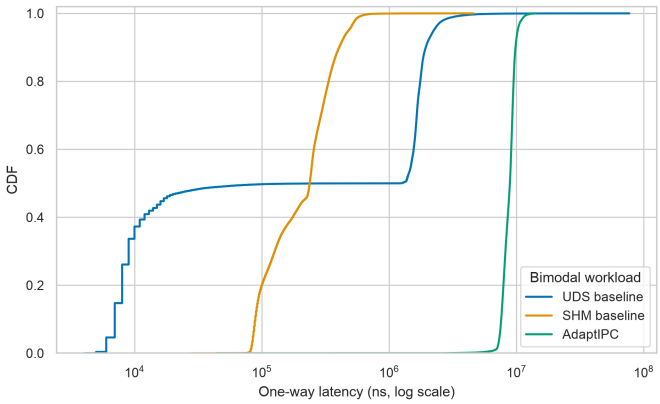

Fig. 6. Empirical CDF of one-way latency under the 100k-iteration bimodal workload. AdaptIPC's distribution is bimodal by construction: control messages ride the low-latency UDS path while bulk payloads ride the high-throughput SHM ring, without application intervention.

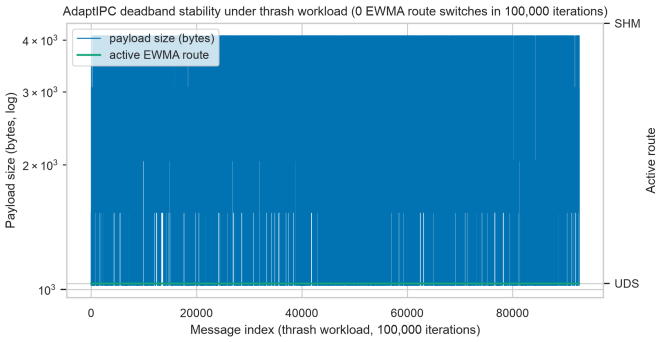

Fig. 7. AdaptIPC engine under the 100,000-iteration thrash workload. The dense vertical banding in the left-axis series is the per-message payload oscillation itself—each message alternates between 1024 B and 4096 B (with intermediate sizes) inside the deadband $\Delta_\tau = \tau_{\text{high}} - \tau_{\text{low}} = 3072$ B; it does not indicate route changes. For rendering efficiency, consecutive duplicate payload-size samples are collapsed into single plotted steps; this collapsing is a plotting artifact only—some duplicates arise specifically because 4096 B frames exceed the datagram ceiling and are escalated from UDS to SHM without moving the sticky hysteresis state (Section IV-C)—and it is not evidence of route or payload change. The right-axis active-route line remains flat at UDS for the entire run: the EWMA state machine performs zero route switches (and hence zero false switches) over all 100,000 iterations, empirically validating the stability proof of Section III-D.

τ_{high}—in particular, any oscillation induced by payloads moving within the deadband. (Per-message escalation of oversized datagrams is a transport-level override and deliberately does not move the EWMA state, so it never counts as a switch.) Under this definition the thrash workload exhibits exactly zero switches—false or otherwise— across all 100,000 iterations, matching the analysis of Section III-D; the sole warm-up transitions appear in the sweep and bimodal workloads (Table I).

A. Limitations

Instrumentation of the router (in-process counters over the full 100,000-iteration bimodal run) exonerates the components one might first suspect. The EWMA update and route decision cost 23 ns per message; the poll(2)-based bounded receive hit its 2 ms timeout on only 29 of 100,000 deliveries, and 94.6% of messages were delivered by the very first ring poll; staging-buffer allocation was eliminated entirely by scatter-gather ring writes. The measured residual latency cost is queueing delay: the shared-memory ring is deliberately deep (64 MB) to decouple producer and consumer for throughput, and under a sustained 2 MB-message backlog the producer can outrun the consumer's drain rate, so a message may wait behind tens of megabytes of buffered traffic (≈4.8 ms median on the bimodal workload). This is an intentional, throughput-oriented tradeoff rather than per-message overhead: the same engine shows 20 μs median latency on the sweep workload once backlog is shallow. Bounding latency via high/low watermark flow control (at some cost to peak throughput) is left for future work. All values in this section come from a single coherent measurement campaign; we observed run-to-run throughput variance of up to ∼3×

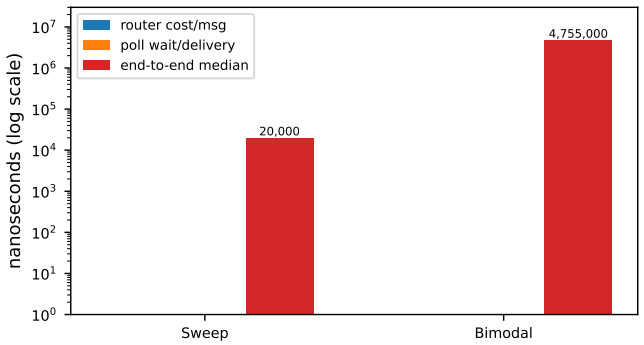

Fig. 8. Measured latency decomposition for the AdaptIPC engine (log scale) on the sweep and bimodal workloads, derived from in-process counters: per-message router cost (EWMA update + route decision), mean receiver poll wait per delivery, and median end-to-end latency. Component costs are two to three orders of magnitude below the end-to-end bimodal median, confirming that the residual latency is queueing delay behind buffered bulk traffic rather than per-message processing overhead.

on the UDS baseline under background system load, so camera-ready numbers should be reported as medians over repeated runs. We explicitly note one consequence: in Table I the Thrash SHM entry (4,515 MB/s) is lower than the corresponding Sweep (6,553 MB/s) and Bimodal (7,387 MB/s) entries, inverting the ordering of Fig. 5. This inversion is measurement variance, not a property of the ring buffer: the thrash workload transfers only 265 MB in 20–85 ms of wall time—too brief to reach steady state—and across seven repeated thrash/SHM measurements throughput spanned 3.1–13.2 GB/s, with the fastest run exceeding every sweep SHM measurement. Short small-message runs are simply far more sensitive to background load than multi-second bulk transfers. Figure 8 visualizes the measured decomposition for the sweep and bimodal workloads.

VI. Conclusion and Future Work

AdaptIPC demonstrates that IPC transport selection can be dynamic, transparent, and provably stable. Combining a C11-atomics-only SPSC shared-memory engine with an EWMA hysteresis classifier yields a raw fast-path throughput of 8.9–21× the Unix Domain Socket baseline measured in the same runs (7.4 GB/s median), while full end-to-end routing sustains 2.2–8.1× UDS throughput; its residual median latency under sustained backlog is buffering-induced queueing delay (Section V-B), not per-message overhead. The engine exhibits exactly zero false route switches over adversarial thrashing workloads—matching our analytical proof—all behind a single API surface that requires no application changes.

Future work proceeds in three directions. First, eBPF kernel bypass: an eBPF hook could steer messages at the kernel boundary, eliminating even the fallback path's syscall cost and enabling routing decisions informed by kernel-observed flow statistics beyond payload size. Second, NUMA-aware multi-producer extension: generalizing the SPSC ring to multi-producer/multi-consumer topologies with per-NUMA-node rings and NUMA-local placement would extend AdaptIPC from pairwise channels to system-wide service meshes, where cross-socket traffic amplifies both copy costs and the benefit of adaptive routing. Third, lazy endpoint negotiation: adapt_init() currently performs eager shared-memory setup for every endpoint—shm_open, ftruncate, and mmap of the ring, a median of 52 μs per endpoint on our machine—regardless of the eventual traffic mix, so channels that turn out to carry only control messages never avoid this cost. Deferring negotiation until a stream is first classified as bulk-heavy (with careful handling of the producer/consumer creation race this introduces) would spare such endpoints the mapping entirely; we leave it for future work.

References

[1] Apple Inc., "Daemons and Services Programming Guide: XPC Services," Apple Developer Documentation.
[2] S. Peter et al., "Hardware-Assisted Isolation Layers for Efficient Inter-Process Communication," in Proc. Workshop on Hardware Assisted Security, 2016.
[3] M. Hicks, J. S. Moore, and S. Nettles, related work on SlipStream-style transparent optimization and interposition of communication layers, 2003–2010.
[4] S. Sridharan, J. Dinan, and D. K. Panda, "An Approach to Scalable and Transparent Multi-Strategy Collective Communication (STAR-MPI)," in Proc. IEEE/ACM SC Workshops, 2008.
[5] M. Herlihy and N. Shavit, The Art of Multiprocessor Programming. Morgan Kaufmann, 2012.
[6] IEEE Std 1003.1, POSIX Realtime: Shared Memory Objects (shm_open), The Open Group.
[7] Eclipse Foundation, "iceoryx2 – the zero-copy data plane," https://iceoryx.io.
[8] The ZeroMQ authors, "ZeroMQ – An open-source universal messaging library," https://zeromq.org.
[9] G. D'Amore et al., "NNG – nanomsg-next-gen: a lightweight, broker-less messaging library," https://nng.nanomsg.org.
[10] J. Axboe, "io_uring: efficient asynchronous I/O via shared submission and completion rings," Linux kernel interface, https://kernel.dk/io_uring.pdf.